

How Onam finds the attack path — and proves it before it alerts you

A reproducible method for turning millions of cloud facts into a short list of verified, priced routes to your crown jewels.

[Graph reasoning](#)[5-domain verification](#)[MITRE ATT&CK](#)[Choke points](#)[FAIR pricing](#)

EXECUTIVE SUMMARY

A finding is noise. A verified path is a decision.

Every cloud scanner produces thousands of findings. The hard problem is not detection — it is knowing which handful of them combine into a route an attacker could actually walk to something that matters, and what that route is worth in dollars. Onam treats that as a graph-reasoning problem, solved the same way every time.

This paper describes the method end to end: how Onam ingests seven clouds read-only, normalizes everything into one property graph, derives the relationships an attacker would exploit, searches for routes from internet-reachable entry points to crown jewels, and — critically — **verifies every edge across five independent security domains before it is ever called a confirmed path**. Each hop is mapped to a MITRE ATT&CK technique; the full set of paths is reduced to its top five choke points; and the resulting exposure is priced with the FAIR model so the output is a board-ready number, not a severity label.

The claim, precisely. Onam does not promise to block attacks at runtime. It promises something more useful for prioritization: a **reproducible, verified, and priced** map of the routes that exist in your cloud right now — computed from read-only telemetry, with a deterministic identity for every finding so results are stable across rescans.

01 The problem with a list

A modern multi-cloud estate generates findings faster than any team can trace them. The industry's default answer is severity: sort by Critical, work down the list. But severity is a property of a finding *in isolation*. It cannot tell you that a "medium" public endpoint, a "low" over-permissive role, and an unencrypted data store are — together — a single four-hop route to your customer records.

Attackers do not read your list. They chain. A route is only as strong as its weakest link, and the link that matters is rarely the one with the scariest CVSS score. To prioritize like a defender who thinks like an attacker, you need to reason over the *relationships* between findings, not the findings themselves.

FIGURE 1 — THE SHIFT

SEVERITY LIST

CRIT · unrelated finding
HIGH · unrelated finding
MED · public endpoint — on a path
LOW · over-priv role — on a path

the two that matter are buried at the bottom

VERIFIED PATH



The same four findings, re-framed. Severity scatters them; the graph connects them into one prioritizable route with a single highest-leverage fix. Illustrative.

02 Ingestion — read-only, agentless, seven clouds

The method starts with telemetry that is safe to collect. Onam connects to each cloud with the lowest standing privilege that permits a security read, and stores no long-lived credentials — only a reference to the role or principal, exchanged for short-lived tokens at scan time.

Cloud	Connection	Privilege posture
AWS	Cross-account IAM role via <code>sts:AssumeRole</code> , unique External ID per tenant	SecurityAudit + ReadOnlyAccess; STS creds $\leq$ 60 min
Azure	Entra service principal	Reader at subscription scope + read-only Graph
Google Cloud	Service account or keyless Workload Identity Federation	6 read-only roles (viewer, securityReviewer ...)
OCI	Dedicated IAM user, API signing key	Policy limited to <code>inspect</code> + <code>read</code>
Alibaba / IBM	RAM read-only · Service ID (Viewer + Reader)	Read-only; API key → short-lived Bearer
Kubernetes	Read-only ServiceAccount, agentless	ClusterRole <code>get / list / watch</code>

Each connection runs a four-stage pipeline — **enumeration** → **enrichment** → **normalization** → **drift** — writing an asset inventory and, crucially, an *asset-relationship* table. The relationships are what make the next step possible.

03 One data model — the property graph

Every engine, for every cloud, emits the **same standardized finding contract** into one canonical findings store. There are no per-engine report tables to reconcile. On top of that store, Onam builds a **Neo4j property graph per tenant**: assets are nodes; typed relationships are edges.

- ◆ **Containment** — an account contains a VPC contains a subnet contains an instance.
- ◆ **Attachment** — a role, policy, security group, or volume attached to a resource.
- ◆ **IAM trust** — who can assume, impersonate, or escalate to whom.
- ◆ **Network reachability** — what can actually reach what, across routes and peering.

Normalization is what lets one graph span seven clouds. A resource's identity is canonicalized to `resource_uid` (rather than an ARN vs OCID), its owner to `account_id` (vs subscription vs project). Every finding carries a `finding_id` that is a **deterministic hash** — the same issue produces the same identifier on every rescan, so paths are stable and remediation can be tracked. A `tenant_id` is a not-null key on every row.

Why a graph, honestly. A security graph is now table stakes — every serious CNAPP markets one. Onam's difference is not that the graph exists; it is what happens to an edge *before* it's confirmed (Section 05), and what the finished path is priced at (Section 07).

04 Deriving the edges an attacker would use

Raw relationships describe how the cloud is built. Attack-path reasoning needs edges that describe how it could be *abused*. Onam loads assets, relationships, and failing findings into the graph, then runs roughly **25 catalog-driven edge derivers** — each one encodes a way an attacker moves: assume-this-role, use-this-key, reach-this-port, read-this-secret, escalate-through-this-policy.

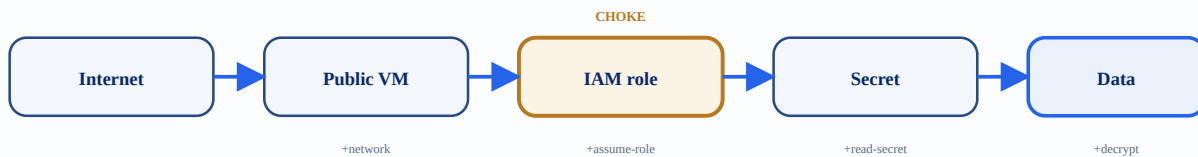
With the abuse-edges in place, the graph is classified at both ends of any route:

- A Entry points** — assets that are internet-reachable, the places an external attacker can actually start.
- B Crown jewels** — sensitive data stores and administrative identities, the places an attacker wants to end.

05 Search — BFS from entry to crown jewel

Onam performs a breadth-first search from every entry point toward every crown jewel, **accumulating attacker capability at each hop**. Reaching a role that can read a secret changes what is reachable next; the search carries that state forward, so a path is evaluated the way an intruder would experience it — as a growing set of capabilities, not a static list of nodes.

FIGURE 2 — CAPABILITY ACCUMULATES ALONG THE WALK



Each hop adds capability (labels under the nodes). The search reasons over the accumulated set, not each node alone. Illustrative.

Verification — five domains before "confirmed"

This is the step that separates a plausible path from a real one. Before any edge is allowed to stand as part of a **CONFIRMED** path, it is verified across **five independent security domains** — never on a single signal. A relationship that looks exploitable from IAM alone but is contradicted by network reachability does not survive. The effect is a dramatically shorter, higher-confidence path list: the routes that pass are the ones worth a human's attention.

06 Mapping to MITRE ATT&CK, then to choke points

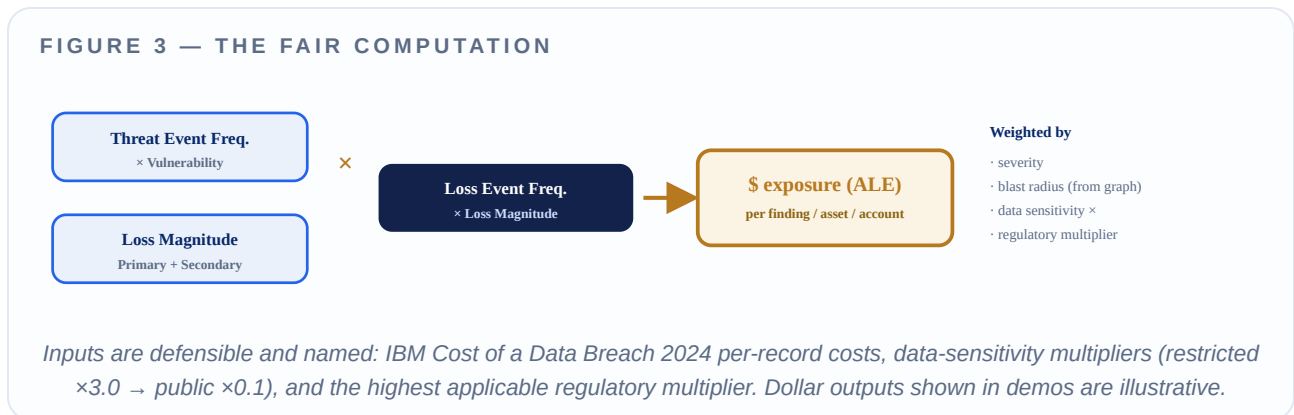
Every hop on a confirmed path is annotated with the **MITRE ATT&CK technique** it represents, producing a technique chain that a threat-informed defender can reason about directly — for example credential access via T1552.005 (cloud instance metadata) leading to valid-accounts use via T1078.004 and data access via T1530. Onam maps to ATT&CK; it does not claim exhaustive coverage of the framework.

Across all confirmed paths, a few nodes recur far more than others. Onam computes the **top five choke points** — single resources whose remediation severs the greatest number of distinct paths — and tells you to triage those first. One well-chosen fix at a choke point can neutralize dozens of routes at once, which is why choke-point ranking, not finding count, drives the work queue.

Severity	Remediation window	Lifecycle
Critical	7 days	Auto-resolves on next PASS; auto-closes after 30 days unseen. Deterministic IDs keep history intact across rescans.
High	30 days	
Medium	90 days	
Low	next cycle	

07 Pricing the path — FAIR as the fourth layer

A verified, ATT&CK-mapped path is still a technical artifact. To make it a business decision, Onam prices it with **FAIR** (Factor Analysis of Information Risk), the open standard for quantifying risk in financial terms.



The result is a dollar exposure per finding, asset, and account — weighted by severity and by the *blast radius the graph already computed*. That is the number a CISO can take to a board, and it is derived, not asserted.

08 End to end — an illustrative walk

Putting the method together on a single representative route:

- 1 Ingest.** Read-only connect surfaces a public-facing VM (entry point) and a customer-records store (crown jewel).
- 2 Derive.** Edge derivers find: the VM's instance role can be assumed; that role can read a secret; the secret decrypts access to the records store.
- 3 Search.** BFS connects internet → VM → role → secret → records, carrying capability forward at each hop.
- 4 Verify.** The four edges are checked across five domains; all hold. The path is marked CONFIRMED.
- 5 Map & rank.** Each hop gets its ATT&CK technique; the *IAM role* is identified as a choke point shared by many other paths.
- 6 Price.** FAIR values the exposure using the records' sensitivity and count. The output: fix one role, cut the most dollars.

09 What this method does not do

Posture, detection, and priced reasoning — not inline blocking. Onam is read-only and agentless by design. It computes and prices the routes that exist; it does not sit in the data path and stop traffic at runtime. Cloud-detection signals come from audit logs (e.g. CloudTrail, Azure Monitor, Kubernetes API), not from an inline enforcement agent. For live blocking, Onam is complementary to a runtime enforcement tool — its role is to tell you which few things to fix so the runtime layer has far less to catch.

We state this plainly because the value of the method depends on trusting it. A path list you can reproduce, verify, and price is worth more than a longer list of things that might be true.

10 Summary

Onam converts read-only, multi-cloud telemetry into one property graph; derives the edges an attacker would abuse; searches for routes from real entry points to real crown jewels; and — before calling anything confirmed — verifies each edge across five independent domains. It maps every hop to MITRE ATT&CK, reduces the whole path set to its five highest-leverage choke points, and prices the exposure with FAIR against named, defensible inputs. The output is short, verified, ranked, and denominated in dollars.

See it on your own cloud. The entire method runs read-only. A connect takes minutes and nothing is installed on a workload — you see your own entry points, paths, choke points, and priced exposure, computed the way this paper describes.